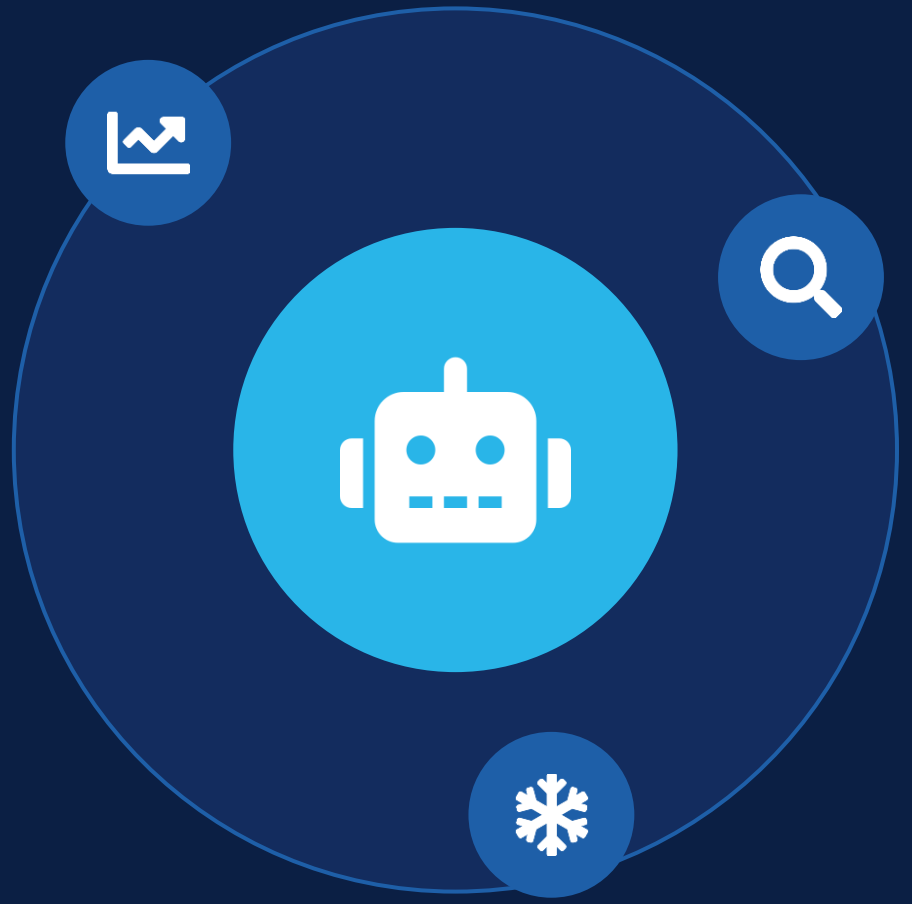
CloudOps AI Copilot

Local-First Incident Detection,
Investigation & AI Analysis

Simulate | Detect | Investigate | Analyze | Persist



PIPELINE

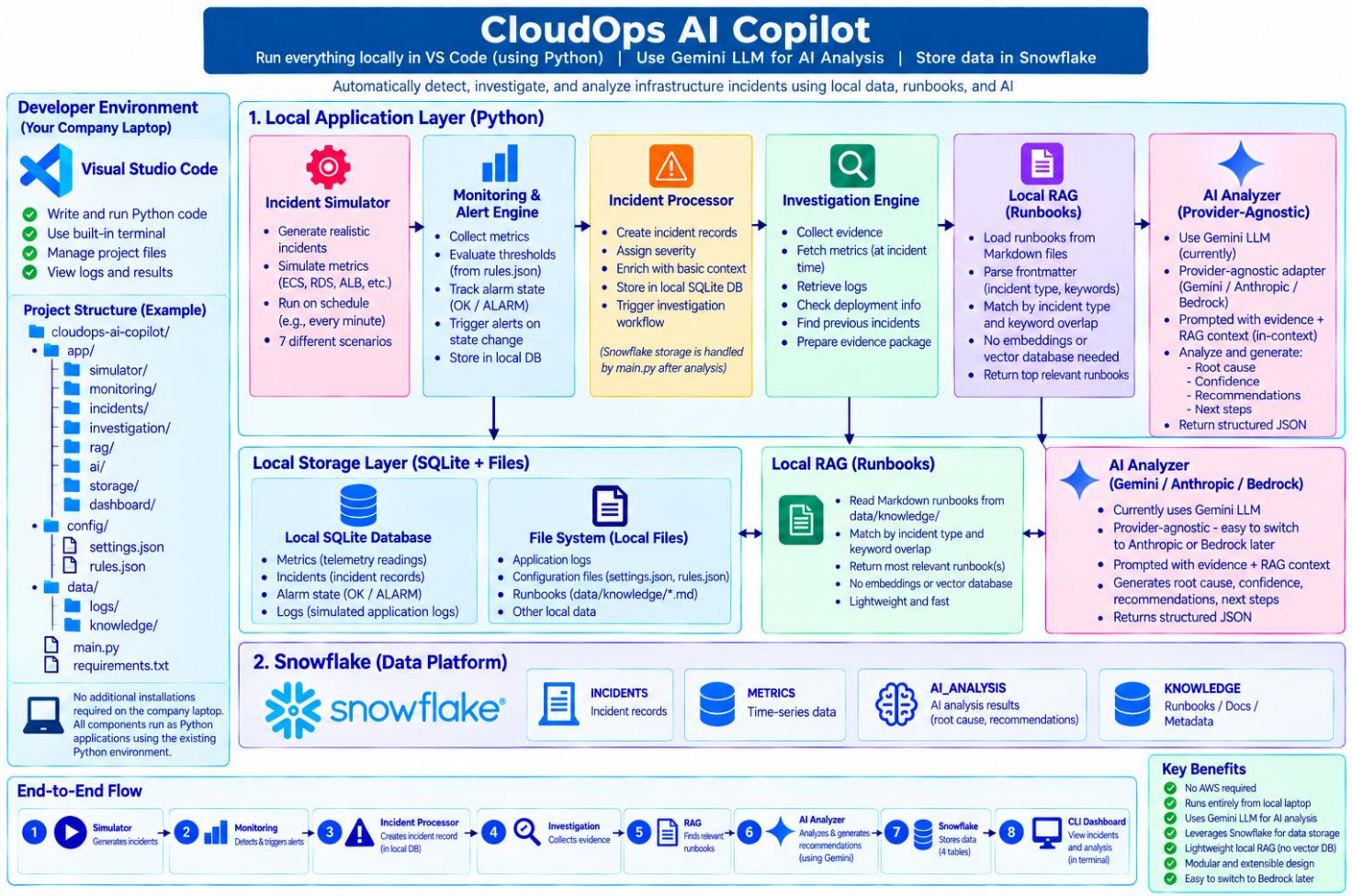
Architecture Flow



Cloud AI provider call

Everything runs locally except the AI model call itself, which goes to a real cloud AI provider.

Solution Architecture



Key benefits

- Runs entirely from local laptop
- Uses Gemini LLM for AI analysis
- Leverages Snowflake for data storage
- Lightweight local RAG (no vector DB)
- Modular and extensible design
- Easy to switch to Bedrock later

DATA GENERATION

Incident Simulator

- Generates realistic synthetic telemetry across 7 scenarios
- 5 tracked metrics
- Manual, random, and continuous scenario generation with reproducible seeded randomization

7 scenarios

- Healthy baseline
- ECS CPU high
- RDS connection exhaustion
- ALB 5xx spike
- Deployment regression
- App latency high
- App latency low

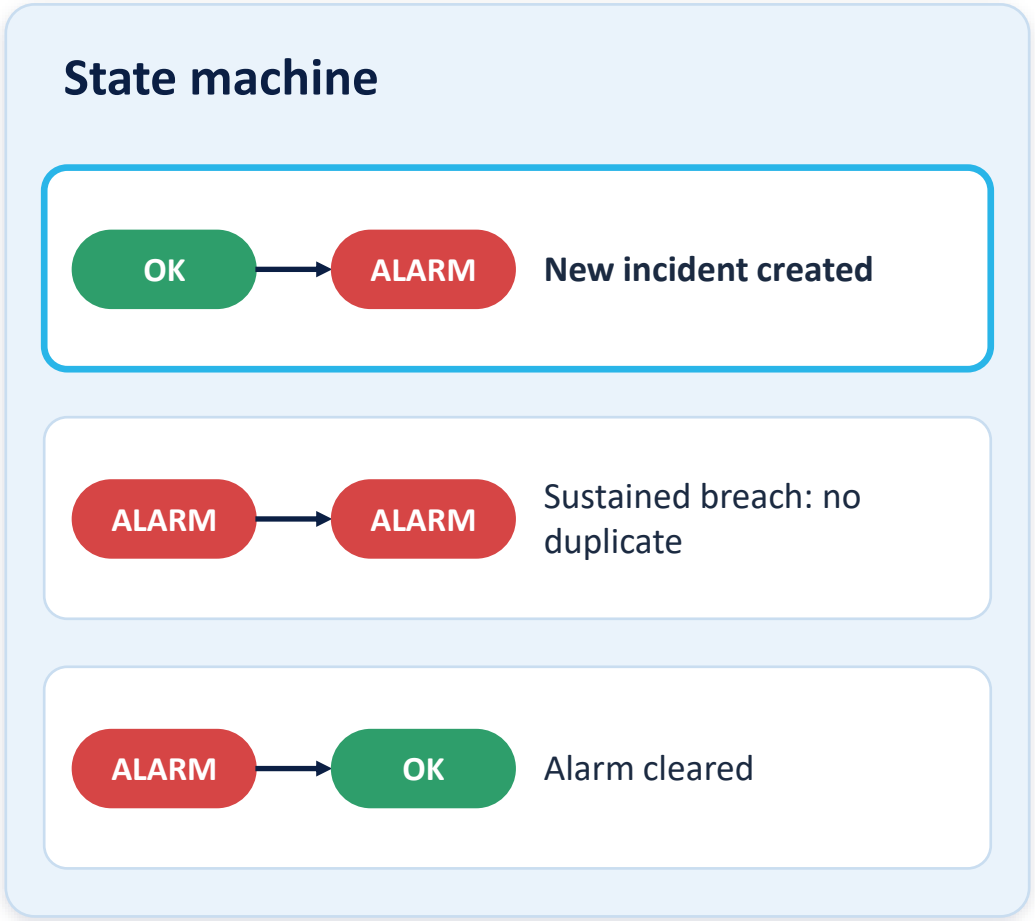
5 tracked metrics

- ECSCPU
- RDSConnections
- ALB5xx
- DeploymentRegression
- AppLatency

DETECTION

Monitoring & Alert Engine

- Configurable thresholds loaded from rules.json, not hardcoded
- Per-alarm state (OK / ALARM) tracked in local SQLite
- State machine: OK→ALARM, ALARM→ALARM, ALARM→OK
- Only an OK→ALARM transition creates a new incident; sustained breaches never duplicate



Incident Processor & Investigation Engine

Incident Processor

`INC-000001`

- Unique sequential IDs (INC-000001, ...)
- Atomic counter avoids collisions

Investigation Engine

- Metrics at the exact incident detection timestamp
- Recent metric history
- Simulated logs correlated to incident type
- Deployment context
- Related past incidents
- Relevant runbooks

Local RAG (Knowledge Base)

- 6 structured Markdown runbooks, one per incident type
- Retrieval by exact incident-type match first, then keyword/tag overlap
- No embeddings or vector database required
- Lightweight and dependency-free

6

Markdown runbooks, one per incident type

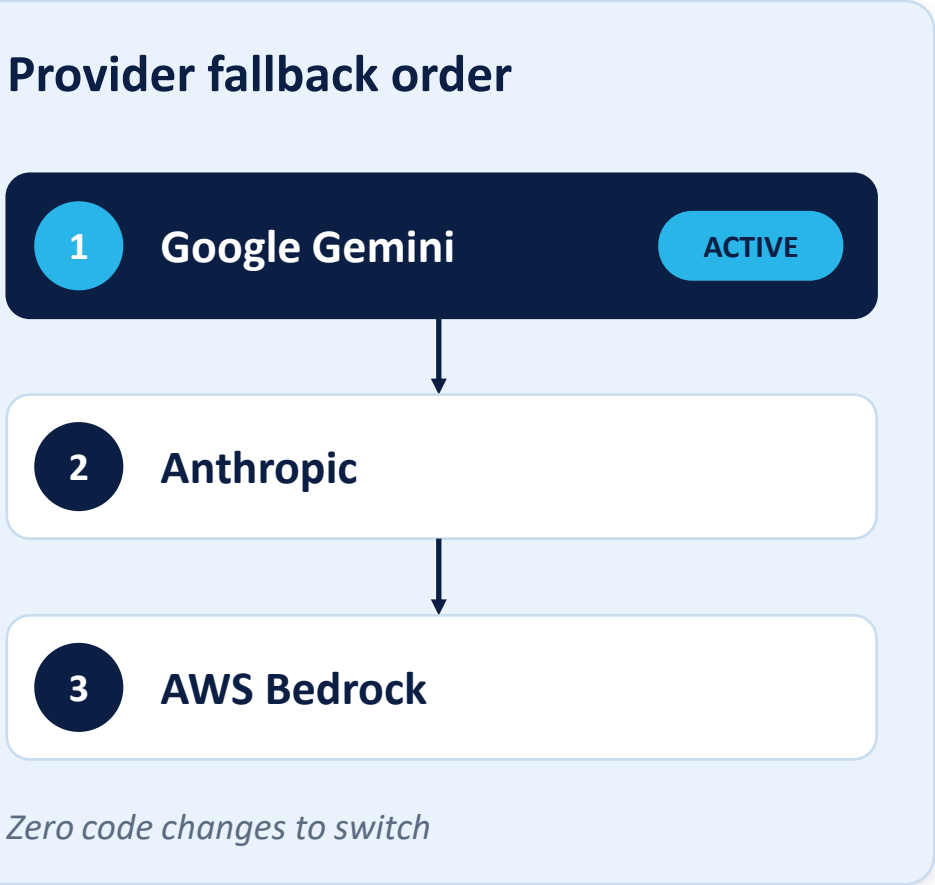
Retrieval order

1. **Exact incident-type match**
2. **Keyword / tag overlap**

AI REASONING

AI Analyzer

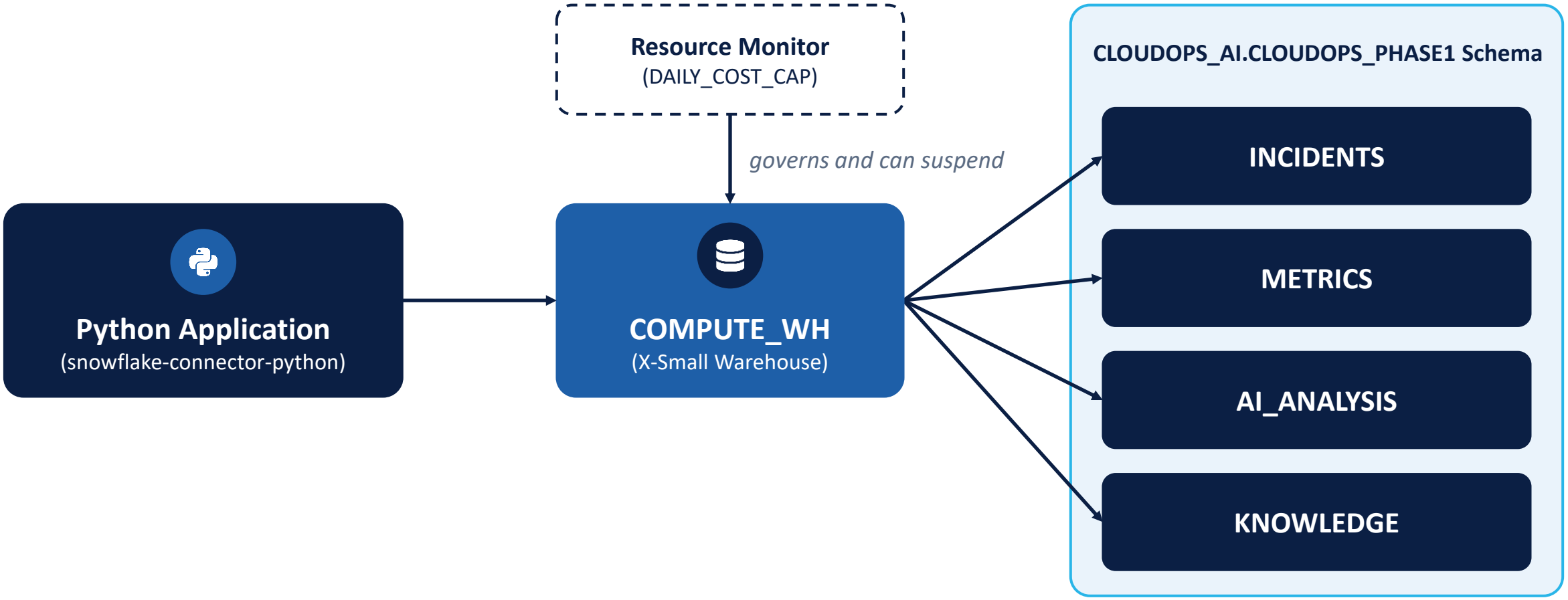
- Builds a structured prompt from the full evidence package + relevant runbook
- Returns root cause, confidence, severity, recommendations, and next steps
- Provider-agnostic adapter tries Gemini, then Anthropic, then AWS Bedrock
- Switching providers needs zero code changes
- Currently serving real AI analysis via Google Gemini



Snowflake: Schema Design

Table	Purpose and contents
INCIDENTS	Incident ID, type, severity, state, reason, service, environment, detection timestamp
METRICS	Every telemetry reading, timestamped
AI_ANALYSIS	Root cause, confidence, severity; recommendations & next steps stored as VARIANT (JSON)
KNOWLEDGE	Runbook library seeded from local Markdown files; tags stored as VARIANT

Snowflake Architecture Diagram (AWS)



Credentials sourced only from environment variables; pipeline degrades gracefully if Snowflake is unreachable.

Snowflake Lessons Learned





PARSE_JSON() in INSERT

Cannot be used inside a plain INSERT ... VALUES; requires INSERT ... SELECT instead



Reserved word: TIMESTAMP

Cannot be used as an unquoted column name



executemany() limits

Cannot rewrite an INSERT ... SELECT; insert row by row when using PARSE_JSON



CREATE TABLE IF NOT EXISTS

Silently preserves a pre-existing table's old schema under that name

Warehouse Cost Management

- Warehouses auto-suspend after a configurable idle period and auto-resume instantly on the next query
- Reduced auto_suspend to 60 seconds to minimize idle billing during active development
- Account-level Resource Monitor (DAILY_COST_CAP) with a daily-resetting credit quota: notify at 50%, hard suspend at 100%
- Credit quota must be a whole number; fractional quotas are not accepted



